

---

# Measuring the Impact of Index Types and Tuning Settings on Query Execution Plans in PostgreSQL and MariaDB

An access-path study across two independently written optimisers

---

**Md. Imtiaz Alam Sajin**

26-94090-2 | 26-94090-2@student.aiub.edu

Supervisor

**Dr. Ashraf Uddin**

23,480 measurements | PostgreSQL 17.1 and MariaDB 10.4.28 | seven table scales | fully reproducible

# A decision nobody sees, and advice nobody measured

---

## **Scan the table, or use an index, and if so which one.**

One decision per query, invisible to the user, worth up to two orders of magnitude in runtime.

The literature attributes bad plans to cardinality misestimation.

That work targets join ordering. The single-relation access-path decision has not been isolated the same way.

## **Four widely recommended practices**

Extended statistics for correlated columns

Lower random\_page\_cost on SSD storage

Add a BRIN index, it costs almost nothing

Enable Multi-Range Read in MariaDB

**Every one is sound in the case it was written for.  
This study measures where each stops being  
sound.**

# Why two engines and not one

---

A single-system study cannot separate a property of cost-based access-path selection from an artefact of one implementation.

---

## **Both engines fail the same way**

Points to a shared design assumption. PostgreSQL and MariaDB were written independently, so agreement is evidence, not coincidence.

## **Only one fails**

An implementation choice, and the other engine shows what a different choice would have bought.

Both systems run identical generated data, identical queries, identical metrics and a matched 128 MB buffer pool.

## METRICS

# Two measures, deliberately kept apart

$R = \text{time of the plan chosen} / \text{time of the fastest plan available}$

---

**Access-path regret** measures the decision. The comparison is against plans the engine really had, forced one at a time, so a slow engine is not penalised for being slow.

**q-error** measures the estimate: the factor by which the predicted row count misses the truth. Reported separately throughout, because the paper's main negative result is that the two come apart.

1.0

the engine made the best  
choice available to it

23,480

measurements across  
both engines

# Symmetric, controlled, and seeded

## Three properties varied independently

Value skew (Zipfian, 4 levels), predicate correlation (5 levels), physical clustering (4 levels), giving 11 dataset configurations.

## Why not TPC-H

A fixed dataset fixes all three at whatever values it happens to have, so no effect can be attributed to a single property. Generating the data also makes the true row count known for every query.

| Table size           | PostgreSQL | MariaDB |
|----------------------|------------|---------|
| 1,000,000            | 2,398      | 1,232   |
| 1,250,000            | 654        | 336     |
| 1,500,000            | 654        | 336     |
| 2,000,000            | 654        | 336     |
| 3,000,000            | 654        | 336     |
| 5,000,000            | 654        | 336     |
| 10,000,000           | 2,398      | 1,232   |
| Configuration sweeps | 6,230      | 5,040   |
| Total                | 14,296     | 9,184   |

## RESULT 1

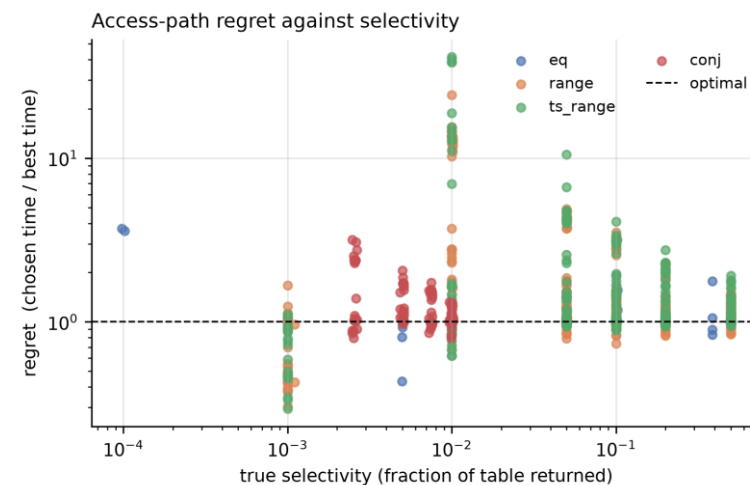
# Selection quality differs by an order of magnitude

| Engine          | Queries | Chose badly | Median regret | Worst case |
|-----------------|---------|-------------|---------------|------------|
| PostgreSQL 17.1 | 130     | 2.3%        | 1.01          | 1.39       |
| MariaDB 10.4.28 | 153     | 66.0%       | 1.34          | 7.91       |

## Read the last two columns together

PostgreSQL is almost always right, and when wrong is wrong by 39%. MariaDB is usually wrong, but its median error is 34%. These are two different failure profiles, not one engine being better at everything, and the rest of the talk separates them.

Measured at 1,000,000 rows. Both engines judged only against plans the other could also have formed.



## RESULT 1, DECOMPOSED

# The two engines fail on different families

| Family    | PG n     | PG wrong    | PG median    | PG q-error  | MDB n    | MDB wrong     | MDB median   | MDB q-error |
|-----------|----------|-------------|--------------|-------------|----------|---------------|--------------|-------------|
| conj      | 19       | 10.5%       | 1.044        | 71.3        | 20       | 45.0%         | 1.067        | 1.00        |
| <b>eq</b> | <b>3</b> | <b>0.0%</b> | <b>1.013</b> | <b>1.00</b> | <b>3</b> | <b>100.0%</b> | <b>6.837</b> | <b>1.29</b> |
| range     | 55       | 1.8%        | 1.013        | 1.01        | 66       | 71.2%         | 1.347        | 1.84        |
| ts_range  | 53       | 0.0%        | 1.001        | 1.01        | 64       | 65.6%         | 1.343        | 1.97        |

**PostgreSQL** errs almost only on conjunctions, and that is exactly where its estimation error is worst, 71.3. Here the textbook story holds.

**MariaDB** estimates conjunctions perfectly at 1.00 and still errs on 45% of them. On equality it is wrong every time, by a median of 6.8x, with a near-correct estimate.

## MECHANISM

# The gap is a difference in plan repertoire

| Family   | PG bitmap | PG index | PG seq | MDB bitmap | MDB index | MDB seq |
|----------|-----------|----------|--------|------------|-----------|---------|
| conj     | 110       | 0        | 0      | 7          | 103       | 0       |
| eq       | 88        | 0        | 0      | 0          | 88        | 0       |
| range    | 88        | 0        | 0      | 0          | 55        | 33      |
| ts_range | 60        | 19       | 9      | 0          | 55        | 33      |

PostgreSQL sorts row locations into physical order before touching the heap, so even a wrong choice reads the disk sequentially. That puts a ceiling on how bad a mistake can be.

**MariaDB has no equivalent enabled. A mistaken index scan becomes random single-row fetches, and there is no ceiling.**

On conjunctions MariaDB's raw access types are ref 103, index\_merge 7. It merges two indexes only at full correlation.

# MariaDB's worst case was not an estimation failure

| Field               | Value                                        |
|---------------------|----------------------------------------------|
| dataset / family    | skew10 / eq                                  |
| true rows           | 101,636 (10.2% of the table)                 |
| estimated rows      | 218,632                                      |
| <b>q-error</b>      | <b>2.151 (near-correct for an optimiser)</b> |
| path chosen         | index scan                                   |
| best path available | sequential scan                              |
| time taken          | 1,969.7 ms                                   |
| time if scanned     | 248.9 ms                                     |
| <b>regret</b>       | <b>7.92x</b>                                 |

It knew roughly how many rows were coming back and still chose to fetch a tenth of the table one row at a time.

Fixing the statistics would not have changed this decision. The cost model, not the estimate, is what ranked the index scan first.

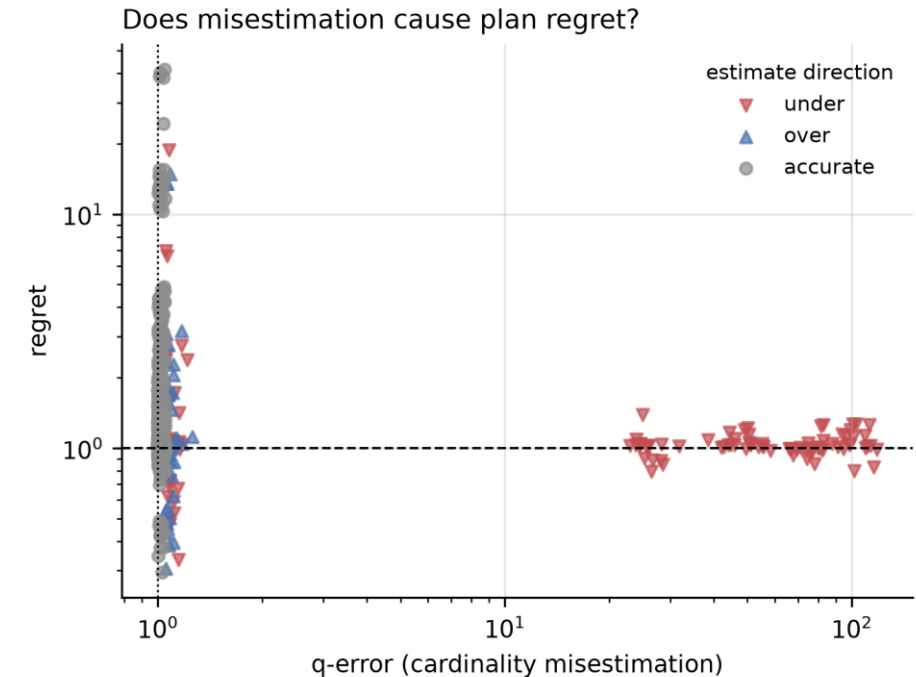
## RESULT 2, THE MAIN NEGATIVE RESULT

# Misestimation does not explain misselection

# 98.2%

of bad access-path choices had an essentially correct row estimate

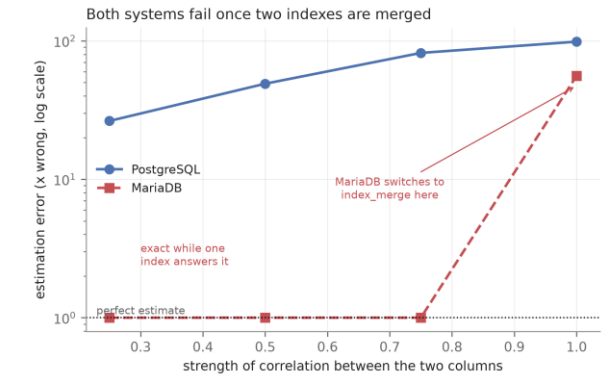
|                           | Value |
|---------------------------|-------|
| Bad choices examined      | 113   |
| Median q-error among them | 1.017 |
| A perfect estimate        | 1.000 |



The remedy most tuning advice reaches for is aimed at a cause that is not operating here.

# Both engines fail once no single index covers both columns

| Correlation | PostgreSQL dependent | independent  | MariaDB dependent | independent |
|-------------|----------------------|--------------|-------------------|-------------|
| 0.00        | 1.13                 | 1.08         | 1.00              | 1.00        |
| 0.25        | 26.39                | 1.37         | 1.00              | 1.00        |
| 0.50        | 49.01                | 2.14         | 1.00              | 1.00        |
| 0.75        | 81.82                | 3.44         | 1.00              | 1.00        |
| <b>1.00</b> | <b>98.65</b>         | <b>98.00</b> | <b>55.84</b>      | <b>1.00</b> |



**The independent columns are the control.** Same queries, uncorrelated data. Both engines stay near 1.0, so the error tracks correlation rather than the query shape.

**Both are accurate only while one index measures both columns directly. Once none does, both multiply two separate guesses and both get it wrong.**

# What generalises, and what belongs to one product

---

## General to cost-based optimisers

### Per-column composition fails under correlation

Both engines accurate to 1.00 with one covering index, both wrong without one.

### Estimation quality and plan quality come apart

Correct row counts accompany 98.2% of PostgreSQL's bad choices.

### Advice ships without its boundary

All four practices tested help in one regime and harm in another.

## Specific to one implementation

### Good selection quality is PostgreSQL's, not the norm

2.3% versus 66.0% on identical data. Not a property of relational optimisers.

### The damage ceiling comes from bitmap scans

MariaDB has no equivalent enabled, so its worst case is unbounded by scale.

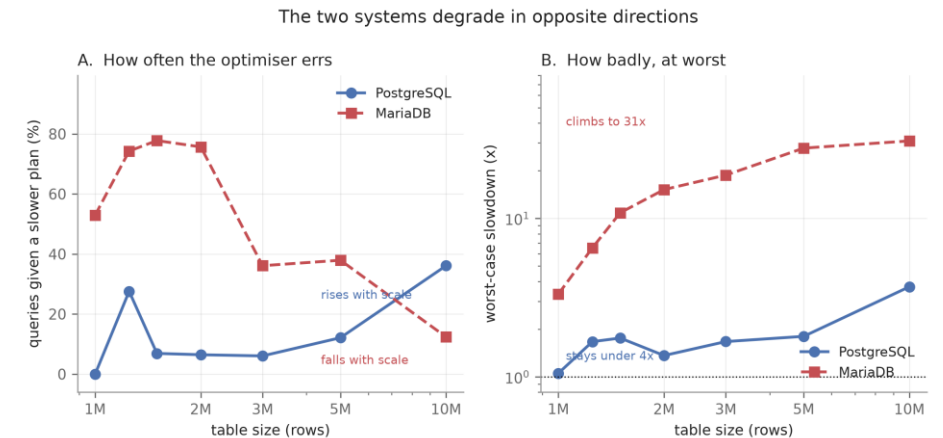
### The BRIN misselection is a PostgreSQL costing defect

MariaDB cannot form the plan at all, so it cannot make this mistake.

## RESULT 4

# The two engines degrade in opposite directions

| Table size        | PostgreSQL<br>wrong | worst case  | MariaDB<br>wrong | worst case   |
|-------------------|---------------------|-------------|------------------|--------------|
| 1,000,000         | 0.0%                | 1.1x        | 52.9%            | 3.3x         |
| 1,250,000         | 27.6%               | 1.7x        | 74.3%            | 6.5x         |
| 1,500,000         | 6.9%                | 1.8x        | 77.8%            | 10.9x        |
| 2,000,000         | 6.5%                | 1.4x        | 75.7%            | 15.2x        |
| 3,000,000         | 6.1%                | 1.7x        | 36.1%            | 18.8x        |
| 5,000,000         | 12.1%               | 1.8x        | 37.9%            | 27.8x        |
| <b>10,000,000</b> | <b>36.1%</b>        | <b>3.7x</b> | <b>12.4%</b>     | <b>30.9x</b> |



**Wrong often but never badly is easier to operate than wrong rarely but catastrophically.**

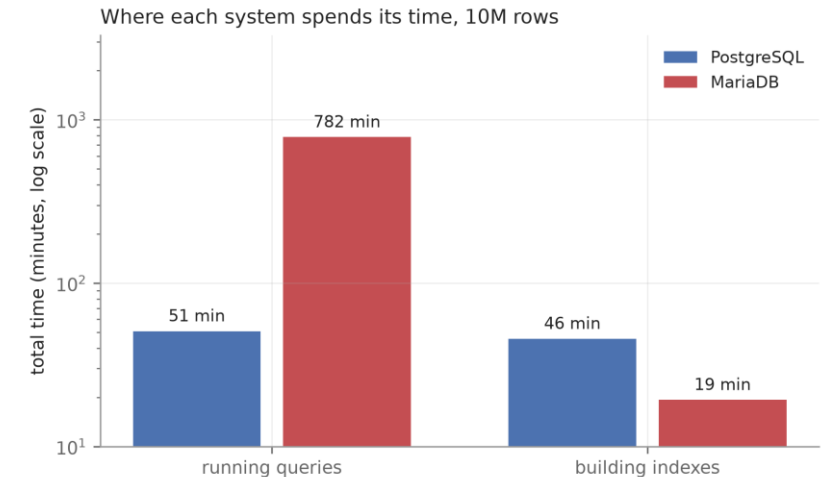
Frequency and severity are separate questions, and here they do not even move in the same direction.

Both engines judged only against plans the other could also have formed, so neither is held to a stricter standard.

## RESULT 5

# Which cost dominates is a property of the engine

|                 | Time running queries | Time building indexes |
|-----------------|----------------------|-----------------------|
| PostgreSQL 17.1 | 50.8 min             | 45.6 min              |
| MariaDB 10.4.28 | 781.8 min            | 19.3 min              |
| <b>Ratio</b>    | <b>15.4x slower</b>  | <b>2.4x faster</b>    |



A study measuring only build time and a study measuring only query time would rank these two engines in opposite orders.

Measured at 10,000,000 rows, matched 128 MB pools, counting only index types both engines support.

# The deficit cannot be configured away

| MariaDB configuration   | Total time     | vs default  | Conjunction q-error |
|-------------------------|----------------|-------------|---------------------|
| ucs3 (statistics depth) | 100.7 s        | 0.87        | 55.84               |
| ucs5                    | 102.1 s        | 0.88        | 55.84               |
| no_index_merge          | 107.0 s        | 0.92        | 55.59               |
| mrr_on_costbased        | 108.4 s        | 0.93        | 55.84               |
| <b>default</b>          | <b>116.1 s</b> | <b>1.00</b> | <b>55.84</b>        |
| mrr_on_sorted           | 121.9 s        | 1.05        | 55.84               |
| <b>mrr_on</b>           | <b>123.8 s</b> | <b>1.07</b> | <b>55.84</b>        |

Enabling MRR makes the workload 7% slower.

Verified that it genuinely engaged rather than being ignored: EXPLAIN reports a rowid-ordered scan and the plan JSON carries the marker, neither of which appears with MRR off.

Statistics depth changes total time by at most 13% and leaves the conjunction error untouched. MariaDB's accuracy there comes from index dives, not from the statistical machinery.

1,000,000 rows, five datasets, 5,040 measurements. No configuration closes the gap with PostgreSQL.

# Why it cannot close it: the buffer is bounded

| mrr_buffer_size                 | 1% selectivity | 5%          | 10%         |
|---------------------------------|----------------|-------------|-------------|
| MRR disabled                    | 1.00           | 1.00        | 1.00        |
| <b>256 KB (shipped default)</b> | <b>1.44</b>    | <b>1.27</b> | <b>1.02</b> |
| 8 MB                            | 0.65           | 1.48        | 1.05        |
| 64 MB                           | 0.66           | 0.97        | 1.00        |

MRR orders identifiers inside a fixed buffer. A bitmap scan materialises the whole qualifying set before touching the heap, so it has no such bound.

**At its best, on the one query class where it helps, MRR recovers about a third of one query's runtime against a cross-system gap of 15.4x.**

## A fourth backfiring practice

"Enable MRR" is reasonable advice on the same grounds as the other three: the mechanism is real and the reasoning sound. At the shipped buffer size it makes the workload slower, and the second setting that would help is not mentioned alongside it.

## Adding an index makes the plan worse

# 194x

slower on a controlled test where the only change was that a BRIN index existed alongside the B-tree

0.34 ms becomes 65.95 ms

| Table size | With BRIN | Without | Worst case |
|------------|-----------|---------|------------|
| 1,000,000  | 73.3%     | 2.3%    | 18.8x      |
| 10,000,000 | 30.1%     | 32.3%   | 2.6x       |

MariaDB cannot form this plan at all, which is how we know it is a costing defect rather than something inherent to having two usable indexes.

Confined to tables that still fit in the buffer pool, which is exactly the size at which BRIN is recommended as a free addition.

# Located in the source, not inferred

```
choose_bitmap_and()    in    optimizer/path/indxpath.c
```

When two indexes can answer the same predicate, PostgreSQL keeps one and discards the other, choosing whichever is cheaper to scan.

That cost is `indextotalcost`, the cost of reading the index alone. It excludes the heap fetches the query must then perform.

**A BRIN index is small by design, so it wins that comparison almost always, and the index thrown away is the faster one.**

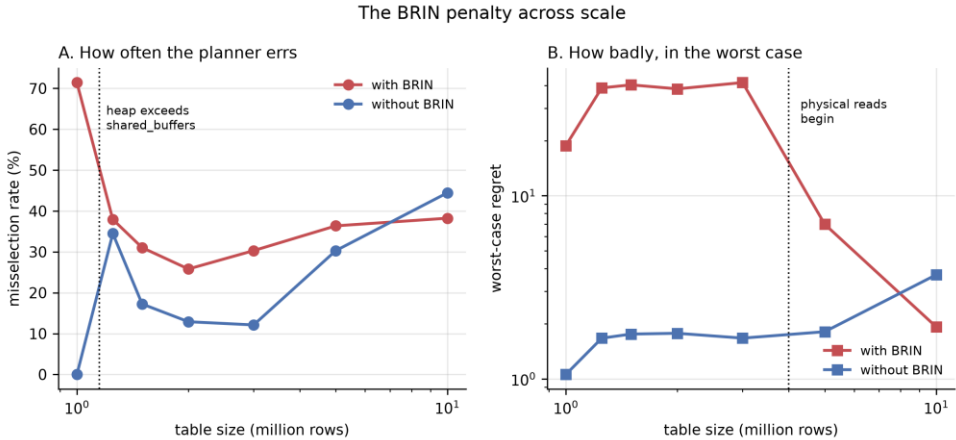
| Index  | Cost used to decide | Actual query time |
|--------|---------------------|-------------------|
| BRIN   | 12.13 (kept)        | 65.95 ms          |
| B-tree | 213.35 (discarded)  | 0.34 ms           |

**The index that costs 17x less to read is the one that runs 194x slower.**

A design consequence of comparing index scan costs in isolation, which is why no setting removes it.

# Frequency and severity move in opposite directions

| Table size       | Heap          | In pool    | Chose badly  | Worst case   |
|------------------|---------------|------------|--------------|--------------|
| <b>1,000,000</b> | <b>112 MB</b> | <b>yes</b> | <b>71.4%</b> | <b>18.8x</b> |
| 1,250,000        | 140 MB        | no         | 37.9%        | 38.8x        |
| 1,500,000        | 168 MB        | no         | 31.0%        | 40.3x        |
| 2,000,000        | 223 MB        | no         | 25.8%        | 38.3x        |
| <b>3,000,000</b> | <b>335 MB</b> | <b>no</b>  | <b>30.3%</b> | <b>41.5x</b> |
| 5,000,000        | 558 MB        | no         | 36.4%        | 7.0x         |
| 10,000,000       | 1,116 MB      | no         | 38.2%        | 1.9x         |



**A two-point scale test would have reversed this conclusion.**

Frequency collapses at the memory boundary; severity doubles there and only subsides much later. The worst damage lands on databases sized two to three times their memory.

## THE PATTERN

# Four practices, each sound, each with an unstated boundary

---

### Extended statistics

for correlated columns

Fixes the estimate 108x

**Query 2.7x slower**

---

### Lower random\_page\_cost

because the disk is an SSD

Default already within 6% of optimal

**Lowering it 2.3x worse**

---

### Add a BRIN index

it is small and cheap

Bad choices 2.3% to 73.3%

**Worst case 194x**

---

### Enable Multi-Range Read

MariaDB, ships switched off

Mechanism genuinely engages

**Workload 7% slower**

**In three of four the advice delivers exactly what it promises, and the plan gets worse anyway.**

# Including one explanation I had to withdraw

---

19

**automated checks before any measurement is accepted**

Generated data validated against PostgreSQL's own pg\_stats, so the generator is checked against something outside my control.

1.79%

**median coefficient of variation above the 1 ms floor**

24.47% below it, which is why that floor exists. Outlying runs are retained, not deleted; the median absorbs up to two in six.

3

**of my own errors found, corrected and reported**

Including a claim I refuted myself: disabling the merge plan left the error at 55.84 against 55.59, so the merge shows the error rather than causing it.

# Four claims

---

**1 The cause is the cost model, not the statistics**

98.2% of bad choices carried correct row estimates. Better statistics cannot repair a comparison that omits heap work, so the fix belongs in costing.

**2 Tuning advice is incomplete without its conditions**

All four practices tested are sound in the case they were written for and harmful outside it. None ships with that boundary stated.

**3 One number cannot describe an optimiser**

How often it errs and how badly it errs move in opposite directions across scale, and in opposite directions between these two engines.

**4 Per-column composition is a shared weakness, not a bug**

Two independently written engines are accurate only while a single index covers both columns, and both fail once none does.

---

# Thank you. Questions?

---

## **Everything is reproducible**

23,480 raw measurements with every repetition retained, seeded generators, the analysis notebook and both write-ups.

## **Md. Imtiaz Alam Sajin**

26-94090-2

Supervisor: Dr. Ashraf Uddin

[github.com/Imtiaz-Sajin/Research-on-Database-Architecture](https://github.com/Imtiaz-Sajin/Research-on-Database-Architecture)